

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Advanced Network

Lab 02: Wireshark Lab (TCP)

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

Academic Year: 2025 / 2026

Seq	Time	Source	Destination	Protocol	Length	Info
81	2.526631	10.131.30.61	128.119.245.12	HTTP		

```

▼ Internet Protocol Version 4, Src: 10.131.30.61, Dst: 128.119.245.12
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    [Total Length: 58375 bytes (reported as 0, presumed to be because of "TC
    Identification: 0x24be (9406)
  ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.131.30.61
    Destination Address: 128.119.245.12
    [Stream index: 6]
  ▼ Transmission Control Protocol, Src Port: 61996, Dst Port: 80, Seq: 94588, A
    Source Port: 61996
    Destination Port: 80
  
```

Q1. What is the IP address and TCP port number used by your client computer?

Source Address: **10.131.30.61**

Source Port: **61996**

Q2. What is the IP address and port number of the server?

Destination Address: **128.119.245.12**

Destination Port: **80**

TCP Basics:

Q3. What are the sequence numbers of the SYN, SYN-ACK, and ACK segments in the three-way handshake?

42	1.767974	10.131.30.61	128.119.245.12	TCP	66	61996 → 80 [SYN] Seq=0
43	1.963796	128.119.245.12	10.131.30.61	TCP	58	443 → 57189 [ACK] Seq=1
44	1.964750	128.119.245.12	10.131.30.61	TCP	70	80 → 61996 [SYN, ACK] S
45	1.964876	10.131.30.61	128.119.245.12	TCP	54	61996 → 80 [ACK] Seq=1

Although packet #43 appears between the SYN and SYN/ACK, it uses different source/destination ports (443 ↔ 57189) and is unrelated. The three-way handshake for the HTTP connection (61996 ↔ 80) consists of packets #42, #44, and #45.

```
▼ Transmission Control Protocol, Src Port: 61996, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 61996
  Destination Port: 80
  [Stream index: 6]
  [Stream Packet Number: 1]
  ▶ [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0      (relative sequence number)
  Sequence Number (raw): 440868314
  [Next Sequence Number: 1      (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
```

Sequence number (raw) of SYN packet #42: **440868314**

```
▼ Flags: 0x002 (SYN)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...0 = Acknowledgment: Not set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  ▶ .... .... ..1. = Syn: Set
```

What identifies this segment as a SYN segment?

The TCP Flags field contains the SYN bit set to 1, and all other control bits (ACK, FIN, RST, PSH, URG) are 0.

```
▼ TCP Option - SACK permitted
  Kind: SACK Permitted (4)
  Length: 2
```

Can the TCP receiver use Selective Acknowledgments (SACK)?

Yes, Under TCP Options, we can see the entry “SACK Permitted”, indicating that both sides support selective acknowledgments.

Q4. What is the sequence number of the SYN-ACK segment sent by gaia.cs.umass.edu to the client? What identifies it as SYN-ACK? What is the value of the Acknowledgement field in the SYN-ACK segment? How did gaia.cs.umass.edu determine that value?

- It is identified as SYN-ACK because both the SYN and ACK flags are set in the TCP header.

- The acknowledgment field in the SYN-ACK packet (#44) is **440868315**, which equals the client's initial sequence number (**440868314**) + 1.
This shows that the server correctly acknowledged the client's SYN segment and that the three-way handshake is proceeding normally.

Q5. What is the sequence number of the TCP segment containing the header of the HTTP POST command? How many bytes of data are contained in the payload (data) field of this TCP segment? Did all of the data in alice.txt fit into this single segment?

46	1.965374	10.131.30.61	128.119.245.12	TCP	657	61996 → 80	[PSH, ACK] Seq=1 Ack=1 Win=
47	1.965620	10.131.30.61	128.119.245.12	TCP	12870	61996 → 80	[ACK] Seq=604 Ack=1 Win=65
52	2.166121	10.131.30.61	128.119.245.12	TCP	27110	61996 → 80	[PSH, ACK] Seq=13420 Ack=1
59	2.334489	10.131.30.61	128.119.245.12	TCP	34230	61996 → 80	[PSH, ACK] Seq=40476 Ack=1
64	2.337285	10.131.30.61	128.119.245.12	TCP	17142	61996 → 80	[PSH, ACK] Seq=74652 Ack=1
66	2.339046	10.131.30.61	128.119.245.12	TCP	2902	61996 → 80	[ACK] Seq=91740 Ack=1 Win=
81	2.526631	10.131.30.61	128.119.245.12	HTTP	58389	POST	/wireshark-labs/lab3-1-reply.htm

```

Transmission Control Protocol, Src Port: 61996, Dst Port: 80, Seq: 1, Ack: 1, Len: 603
  Source Port: 61996
  Destination Port: 80
  [Stream index: 6]
  [Stream Packet Number: 4]
  ▶ [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 603]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 440868315

```

TCP segment containing HTTP POST header: packet #46 (first data-carrying segment).

Sequence number (relative): 1

Sequence number (raw): **440868315**

Payload bytes in this segment: tcp.len = **603 bytes**.

Did alice.txt fit in this single segment?

No. The file is transferred across multiple TCP segments; Wireshark reassembles the POST from packets 46, 47, 52, 59, 64, 66 into the reconstructed HTTP POST shown in packet #81 (total POST length reported ~ 58389 bytes), so the single 603-byte segment only contains the start of the POST header/body.

Q6. At what time was the first segment in the data-transfer part of the TCP connection sent?

```

[Timestamps]
[Time since first frame in this TCP stream: 197.400000 milliseconds]

```

Time sent: **197.4 milliseconds**

At what time was the ACK for this first data-containing segment received?

```

[Timestamps]
[Time since first frame in this TCP stream: 397.975000 milliseconds]

```

The ack was in packet #48, Time sent is: **397.975 milliseconds**

What is the RTT for this first data-containing segment?

$RTT1 = T1_ack - T1_sent = 397.975 - 197.4 = \mathbf{200.575 \text{ milliseconds}}$

What is the RTT value the second data-carrying TCP segment and its ACK?

$RTT2 = T2_ack - T2_sent = 397.975 - 197.646 = \mathbf{200.329 \text{ milliseconds}}$

What is the EstimatedRTT value?

$EstimatedRTT = (1 - 0.125) \times RTT1 + 0.125 \times RTT2 = 0.875 \times 200.575 + 0.125 \times 200.329 = 175.503 + 25.041 = \mathbf{200.544ms}$

Q7. What is the length (header plus payload) of each of the first four data-carrying TCP segments?

first four data-carrying segments are packets 46, 47, 52, 59:

tcp.stream == 6 && tcp.len > 0 && ip.src == 10.131.30.61					
No.	Time	Source	Destination	Protocol	Length
46	1.965374	10.131.30.61	128.119.245.12	TCP	657
47	1.965620	10.131.30.61	128.119.245.12	TCP	12870
52	2.166121	10.131.30.61	128.119.245.12	TCP	27110
59	2.334489	10.131.30.61	128.119.245.12	TCP	34230

Packet 46 length = **657 bytes**

Packet 46 length = **12870 bytes**

Packet 46 length = **27110 bytes**

Packet 46 length = **34230 bytes**

Q8. What is the minimum amount of available buffer space advertised to the client by gaia.cs.umass.edu among these first four data-carrying TCP segments? Does the lack of receiver buffer space ever throttle the sender for these first four data-carrying segments?

Window: 498
[Calculated window size: 63744]

For each server ACK I read the Window value in the packet list/ TCP header (Wireshark shows the calculated window in bytes); the smallest value observed among the first four ACKs is **63,744 bytes** (packet 48).

Does receiver buffer space throttle the sender for the first four data-carrying segments?

No, The smallest advertised window among the ACKs for the first four data segments is 63,744 Bytes, which is far larger than the sizes of the individual segments being sent (603 B, 12,816 B, 27,056 B, 34,176 B).

Q9. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

657	61996 → 80	[PSH, ACK]	Seq=1	Ack=1	Win=65280	Len=603	[TCP PDU reassembled in 81]
12870	61996 → 80	[ACK]	Seq=604	Ack=1	Win=65280	Len=12816	[TCP PDU reassembled in 81]
58	80 → 61996	[ACK]	Seq=1	Ack=604	Win=63744	Len=0	
58	80 → 61996	[ACK]	Seq=1	Ack=6300	Win=75520	Len=0	
58	80 → 61996	[ACK]	Seq=1	Ack=9148	Win=81280	Len=0	
58	80 → 61996	[ACK]	Seq=1	Ack=13420	Win=86912	Len=0	
27110	61996 → 80	[PSH, ACK]	Seq=13420	Ack=1	Win=65280	Len=27056	[TCP PDU reassembled in 81]
58	80 → 61996	[ACK]	Seq=1	Ack=16268	Win=88320	Len=0	
58	80 → 61996	[ACK]	Seq=1	Ack=20540	Win=86912	Len=0	
58	80 → 61996	[ACK]	Seq=1	Ack=23388	Win=88320	Len=0	

I checked all client-to-server TCP data packets in the stream (packets #46, #47, #52, #59, etc.). Their sequence numbers increase normally and Wireshark does not label any packet as “[TCP Retransmission]” or “[TCP Fast Retransmission].”

Therefore, there are no retransmitted segments in the trace.

Q10. How much data does the receiver typically acknowledge in an ACK among the first ten data-carrying segments sent from the client to gaia.cs.umass.edu? Can you identify cases where the receiver is ACKing every other received segment among these first ten data-carrying segments?

Client Segment	Packet #	Seq (rel)	tcp.len	Time Sent (ms)	ACK Packet #	ACK (cumulative)	Time ACK (ms)
1	46	1	603	197.400	48	604	397.975
2	47	604	12,816	197.646	51	13420	397.975

Among the first ten data-carrying TCP segments sent from the client to gaia.cs.umass.edu, the receiver typically acknowledges the cumulative number of bytes received (not each segment individually).

For the first two data segments (packets #46 and #47), the server sends a single cumulative ACK (Ack = 13420) at 397.975 ms, which acknowledges both segments together. This shows the receiver’s delayed ACK behavior, where it acknowledges every other received segment instead of every single one.

Q11. What is the throughput (bytes transferred per unit time) for the TCP connection?

Throughput = Total bytes transferred / duration

```
▼ [Timestamps]
[Time since first frame in this TCP stream: 957.447000 milliseconds]
[Time since previous frame in this TCP stream: 0.000000000 seconds]
```

Duration = Time of the last ack sent by the server - Time of the first packet sent by the client =
Time of packet #94 - Time of packet #46 = 957.447 - 197.4 = **760.047 ms = 0.760047 s**

```
▼ [7 Reassembled TCP Segments (152922 bytes): #46(603),
  [Frame: 46, payload: 0-602 (603 bytes)]
  [Frame: 47, payload: 603-13418 (12816 bytes)]
  [Frame: 52, payload: 13419-40474 (27056 bytes)]
  [Frame: 59, payload: 40475-74650 (34176 bytes)]
  [Frame: 64, payload: 74651-91738 (17088 bytes)]
  [Frame: 66, payload: 91739-94586 (2848 bytes)]
  [Frame: 81, payload: 94587-152921 (58335 bytes)]
  [Segment count: 7]
  [Reassembled TCP length: 152922]
  [Reassembled TCP Data [...]: 504f5354202f7769722657368]
```

Total bytes transferred = **152922 bytes**

Throughput = 152922 / 0.760047 = 201200.715 bytes per second

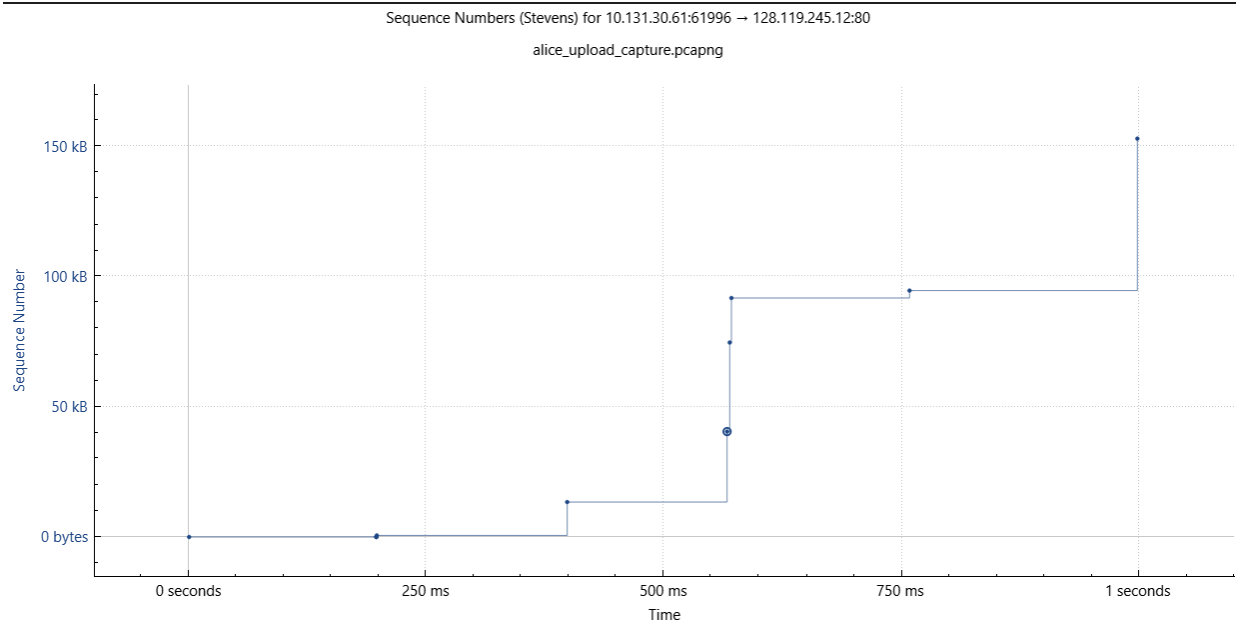
The three-way handshake is excluded from the measurement since no application data is transferred during that phase. The throughput represents the actual data transfer rate for the HTTP POST.

TCP congestion control in action:

Q12. Figure 6 shows clear “fleets” of packets (bursts) sent from the client toward the server. The early fleets grow rapidly in size between successive RTTs, which is characteristic of TCP’s **slow start** (exponential increase of cwnd). After a few fleets the growth rate becomes steadier and more linear, indicating a transition into **congestion-avoidance**. The plot does not show abrupt downward drops or retransmission markers in the example figure, so the behavior illustrated is slow start followed by congestion-avoidance.

Q13. The fleets in Figure 6 exhibit a periodicity approximately equal to one RTT. Measured intervals between successive fleets in the figure are roughly constant and correspond to the RTT scale shown on the horizontal axis. Thus each fleet represents a cwnd-limited transmission that occurs once per RTT, the fleet period $\approx$ RTT.

Q14.



Time–Sequence (Stevens) graph for TCP stream. Four fleets visible at approximately 0.2 s, 0.4 s, 0.6 s, and 1.0 s, showing exponential-to-linear growth transition (slow start → congestion avoidance).

- Measured RTT (from Q6): $RTT_1 = 200.575 \text{ ms}$
- Measured fleet times (approx): **0.20 s, 0.40 s, 0.60 s**
- Fleet period $\approx 0.20 \text{ s} \approx RTT$

For my trace the Time–Sequence (Stevens) graph indicates that TCP starts in **slow start** and then transitions to **congestion avoidance**. The early fleets show rapidly increasing amounts of data sent per RTT (consistent with exponential cwnd growth in slow start). Later fleets grow more slowly and the slope becomes steadier, indicating the sender has entered congestion-avoidance. No retransmission-driven drops are visible in the graph.

The fleets repeat with a period of approximately **0.20 s ($\approx 200 \text{ ms}$)**. Measured fleet times are roughly 0.20 s, 0.40 s, and 0.60 s (intervals $\approx 0.20 \text{ s}$), which matches the RTT measured from the trace (**$RTT \approx 200.575 \text{ ms}$**). Thus each fleet corresponds to sending about one congestion-window's worth of data per RTT.